Revision — 2.3.1 Algorithms

OCR H446 — one-page revision summary

Must know

- **Big O** describes the **worst-case** growth of **time** (operations) or **space** (memory) as input size **n** grows. Drop constants and lower-order terms: `4n² + 200n + 9` → **O(n²)** (dominant term wins).
- Growth order (best → worst): **O(1)** constant → **O(log n)** logarithmic → **O(n)** linear → **O(n log n)** linearithmic → **O(n²)** polynomial/quadratic → **O(2ⁿ)** exponential. **Tractable** = polynomial time or better; **intractable** = exponential or worse.
- **Linear search — O(n).** Checks each element in turn; works on **unsorted** data. Use when the list is small or unsorted.
- **Binary search — O(log n).** Checks the middle of a **sorted** list and discards the half that cannot contain the target. **Requires sorted data** — quote this precondition. Use for large, sorted lists.
- **Bubble sort — O(n²).** Repeatedly compares adjacent pairs and swaps; largest "bubbles" to the end each pass. Best case **O(n)** only **with a swap flag** on already-sorted data. Simple to code but slow — use only for tiny lists.
- **Insertion sort — O(n²).** Builds a sorted front section, inserting each next item into place. Best case **O(n)** on nearly-sorted data. Good for small or nearly-sorted lists.
- **Merge sort — O(n log n) in all cases.** Divide-and-conquer: split in half recursively, then merge sorted sublists. **Guaranteed O(n log n)** but needs **O(n) extra space**. Use when consistent performance matters.
- **Quick sort — O(n log n) average, O(n²) worst.** Divide-and-conquer: pick a **pivot**, partition (smaller left / larger right), recursively sort each side. Worst case with a poor pivot (e.g. already-sorted data). Fast in practice, in-place (**O(log n)** space).
- **Stack (LIFO)** and **queue (FIFO)** operations are **O(1)**. **DFS uses a stack** (or recursion); **BFS uses a queue**.
- **Tree traversals: pre-order** (Node→Left→Right, copy a tree), **in-order** (Left→Node→Right, outputs a **BST in ascending order**), **post-order** (Left→Right→Node, delete a tree / evaluate RPN).
- **Path-finding:** Dijkstra uses **g(n)** only (uninformed, shortest path to all nodes); **A*** uses **f(n) = g(n) + h(n)**, adding an **admissible heuristic** h(n) estimating remaining cost to the goal — it directs the search and usually explores fewer nodes to reach a single goal.

Key terms

Term	Definition
Algorithm	A precise, finite sequence of steps to solve a problem
Big O notation	How an algorithm's time/space requirement grows with input size n (worst case)
Time complexity	Number of operations as a function of input size
Space complexity	Memory required as a function of input size
Tractable / Intractable	Solvable in polynomial time or better / only exponential or worse
Divide and conquer	Break into subproblems, solve, then combine (merge/quick sort, binary search)
Pivot	The element chosen in quick sort to partition the list
Stable sort	Preserves the relative order of equal elements
Heuristic	A rule-of-thumb estimate; in A*, the estimate h(n) of remaining cost to the goal
Admissible heuristic	One that never overestimates the true remaining cost → guarantees A* optimality
Traversal	Visiting every node of a structure in a defined order
g(n) / h(n) / f(n)	Actual cost so far / heuristic estimate to goal / estimated total (g + h)

Worked example / how to — Big O comparison of the standard algorithms (double-checked)

Algorithm	Best	Average	Worst	Space	Stable?	Precondition
Linear search	O(1)	O(n)	O(n)	O(1)	—	none (works on unsorted)
Binary search	O(1)	O(log n)	O(log n)	O(1) iter / O(log n) rec	—	**list must be sorted**
Bubble sort	O(n)*	O(n^2)	O(n^2)	O(1)	Yes	—
Insertion sort	O(n)	O(n^2)	O(n^2)	O(1)	Yes	—
Merge sort	O(n log n)	O(n log n)	O(n log n)	**O(n)**	Yes	—
Quick sort	O(n log n)	O(n log n)	**O(n^2)**	O(log n)	No	—

* Bubble best case O(n) only **with a swap flag** on already-sorted data; otherwise O(n^2).

Key contrasts: **binary search beats linear search** (O(log n) vs O(n)) but needs sorted data. **Merge sort guarantees O(n log n)** at the cost of **O(n) extra space**, whereas **quick sort** is usually as fast and in-place but degrades to **O(n^2)** with a poor pivot — a classic time-vs-space and average-vs-worst trade-off.

Exam tips

- **Binary search is O(log n) and REQUIRES sorted data** — always state the precondition.
- **Always justify** a Big O, e.g. "O(n^2) because the nested loop makes up to n comparisons for each of n passes."
- **Quick sort worst case is O(n^2); merge sort guarantees O(n log n) but uses O(n) extra space** — name the trade-off.

- **DFS uses a stack; BFS uses a queue**, and **in-order traversal of a BST outputs values in ascending order** — do not mix these up.